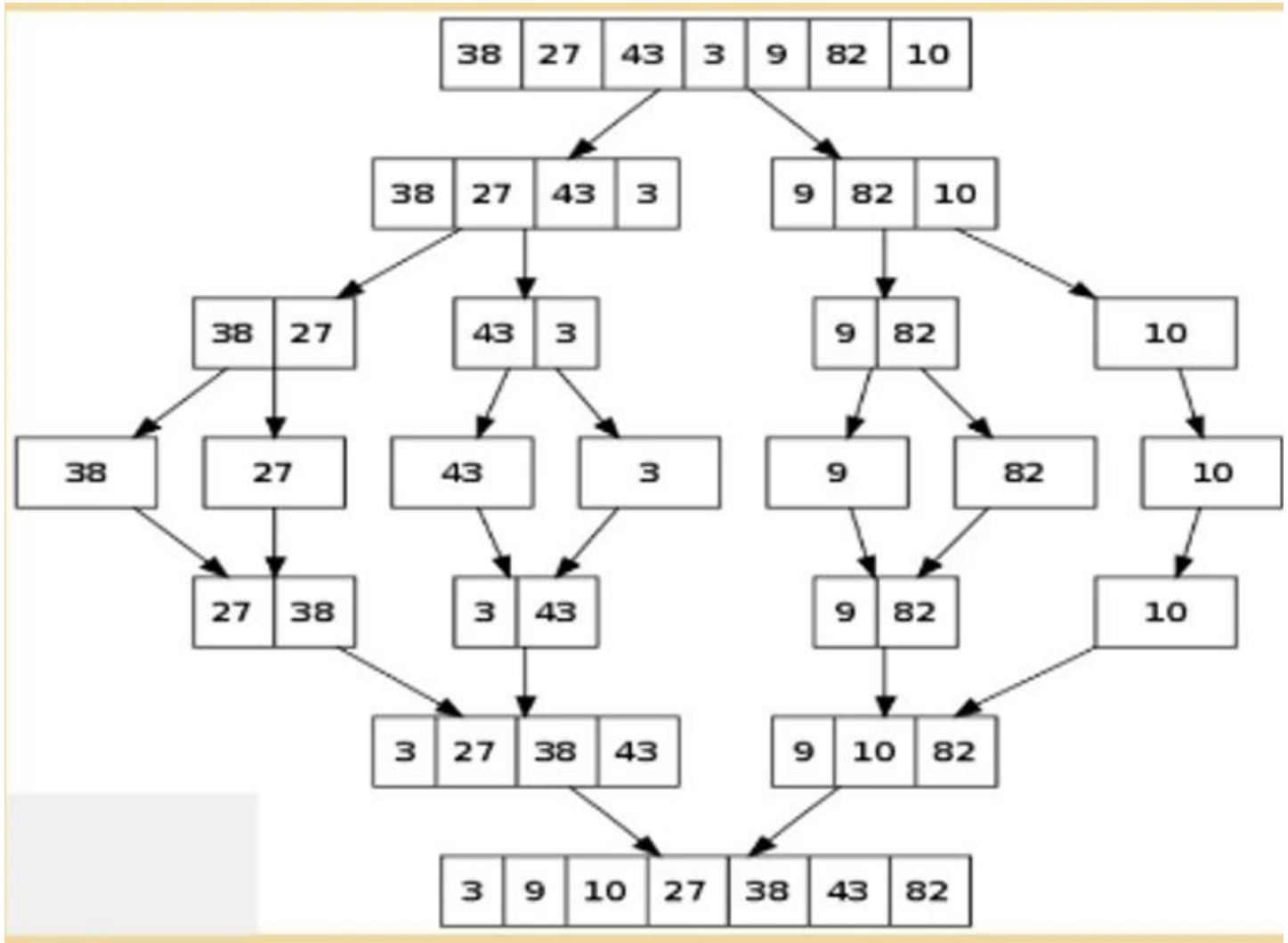
Q and A

Lecture 8

Queston 1

- Consider the following data set:
- 38,27,43,3,9,82,10
- Using a suitable diagram, explain how to sort the above dataset using the **Merge Sort algorithm**.
- Write C code to implement the Merge Sort algorithm

Example – Merge Sort



C Code – Merge sort

```
#include <stdio.h>
// Function to merge two halves
void merge(int arr[], int left, int mid, int right) {
    int n1 = mid - left + 1;  // Size of left half
    int n2 = right - mid;     // Size of right half
    // Temporary arrays
    int L[n1], R[n2];
    // Copy data to temp arrays
    for (int i = 0; i < n1; i++)
        L[i] = arr[left + i];
    for (int j = 0; j < n2; j++)
        R[j] = arr[mid + 1 + j];
```

C Code – Merge sort Cont..

```
// Merge the two halves back into arr[]
  int i = 0, j = 0, k = left;
  while (i < n1 && j < n2) {
    if (L[i] <= R[j]) {
      arr[k] = L[i];
      i++;
    } else {
      arr[k] = R[j];
      j++;
    }
    k++;
  }
}
```

C Code – Merge sort Cont..

```
// Recursive merge sort
void mergeSort(int arr[], int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;
        // Sort first and second halves
        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);
        // Merge the sorted halves
        merge(arr, left, mid, right);
    }
}
```

C Code – Merge sort Cont..

```
// Utility function to print an array
void printArray(int arr[], int size) {
    for (int i = 0; i < size; i++)
        printf("%d ", arr[i]);
    printf("\n");
}
```

C Code – Merge sort Cont..

```
// Main function
int main() {
    int arr[] = {38, 27, 43, 3, 9, 82, 10};
    int size = sizeof(arr) / sizeof(arr[0]);
    printf("Original array: \n");
    printArray(arr, size);
    mergeSort(arr, 0, size - 1);
    printf("Sorted array: \n");
    printArray(arr, size);
    return 0;
}
```

Question 2

a)Sort the following dataset using the **Quick Sort algorithm**:
25, 57, 48, 37, 12, 92, 86, 33

- **Note:** Use the **first element** as the pivot at all stages.

b) write pseudocode algorithm for the quick sort

Sorting Process

- Eg : 25,57,48,37,12,92,86,33
- If we choose x[0] as pivot value and store it in position j
- Then 1st element 25 is placed in its proper position
- 12,**25**,57,48,37,92,86,33
- Nothing need to be done to sort the first of these arrays: file of one element is already sorted.
- To sort the 2nd sub array/list , the process is repeated and the sub array is further sub-divided.

Sorting Process

- (12),**25**,(57,48,37,92,86,33)
- If we choose 57 as a pivot then
- (12),**25**,(48,37,33),57,(92,86)
- If we choose 48 and 92 as pivot.
- (12),**25**, (37,33),48,57,92,(86)
- If we select 37 as pivot
- 12,25,33,48,57,86,92
- The final array/list is sorted.

Quicksort-pseudo code algorithm

- Quicksort Algorithm

```
//Sort the array elements A[first] throught A[last]
Input: Array A, int first, int last
Output:
quicksort(A, first, last){
   if(first < last){
        Choose a pivot
        Partition the array about the pivot
        pivotIndex = index of the pivot
        quicksort(A, first, pivotIndex-1)
        quicksort(A, pivotIndex+1, last)
   }
}
```

Partition Algorithm

- Two references **up** and **down** are moved towards each other in the following fashion.
- **Algorithm**
- **Step 1**: Repeatedly increase the pointer **down** by one position until **x[down] >a**
- **Step 2**: Repeatedly decrease the pointer **up** by one position until x[up] =< a
- **Step 3**: if up > down then interchange **x[down] with x[up]**
- The process is repeated until the condition in step 3 fails (ie up =< down) at that point **x[up]** is interchanged with **x[lb]** (which equal to a), whose final position was sought and **j** is set to **up**.

Question 3

- Apply the above portion algorithm portion the following data set:
- 25 57 48 37 12 92 86 33

Partition Process

down → 25 57 48 37 12 92 86 33 ← up 1st step

25 down → 57 48 37 12 92 86 33 ← up 1st step

25 down → 57 48 37 12 92 86 33 <--up 1st step fail

25 down → 57 48 37 12 92 86 <--up 33 2nd step

25 down → 57 48 37 12 92 <--up 86 33 2nd step

25 down → 57 48 37 12 <--up 92 86 33 2nd step fail

25 down → 12 48 37 57 <--up 92 86 33 ******transition

Partition Process

25 12 down → 48 37 57 <--up 92 86 33 ******

25 12 down → 48 37 57 <--up 92 86 33 1st step

25 12 48 down → 37 57 <--up 92 86 33 1st step fail

25 12 48 down → 37 57 <--up 92 86 33 2nd step

25 12 48 down → 37 57 <--up 92 86 33 2nd step

25 12 48 down → 37 <--up 57 92 86 33 2nd step

25 12 48 up down 37 57 92 86 33 2nd step

25 12 ← up 48 ← down 37 57 92 86 33 2nd step fail

(12) 25 (48 37 57 92 86 33) **********transition

Question 4

- Sort the following numbers using the Buket sort algorithm
- Numbers are :**0.78, 0.17, 0.39, 0.26, 0.72, 0.94, 0.21, 0.12, 0.23, 0.68**
- Find the Worst-Case Time Complexity of the bucket sort

Question 4

- Sort the following numbers using the Buket sort algorithm
- Numbers are :**0.78, 0.17, 0.39, 0.26, 0.72, 0.94, 0.21, 0.12, 0.23, 0.68**
- **Step 1:** Create an array of size 10, where each slot represents a bucket.

Question 4

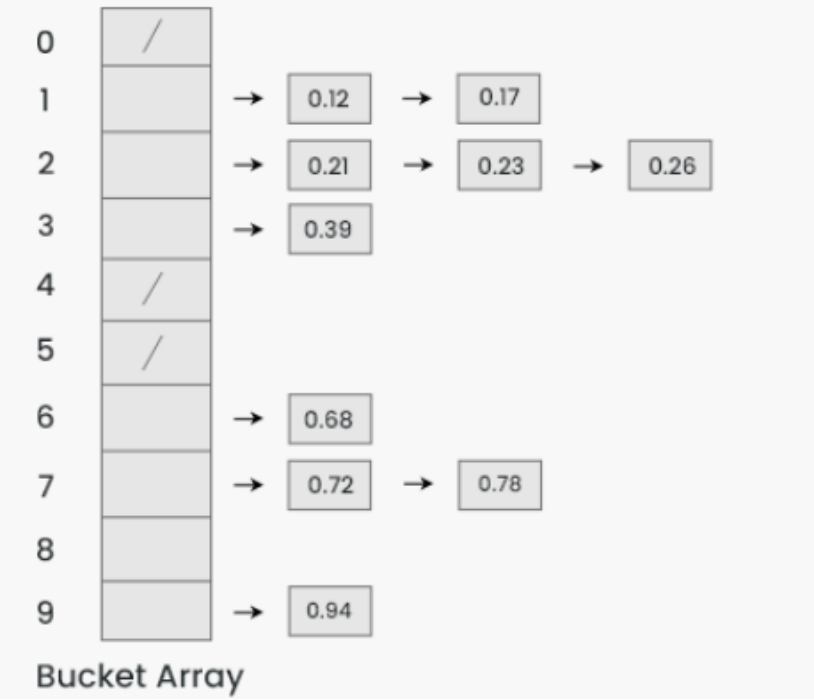
- **Step 2:** Insert elements into the buckets from the input array based on their range. (divide into 10 buckets)

For multiple elements on same bucket, Linked List Data Structure will be used

First bucket 0.01 to 0.09
Second bucket 0.10 to 0.19
And soon

Question 4

- **Step 3:** Sort the elements within each bucket.
- Apply a stable sorting algorithm (e.g., Insertion Sort) to sort the elements within each bucket.

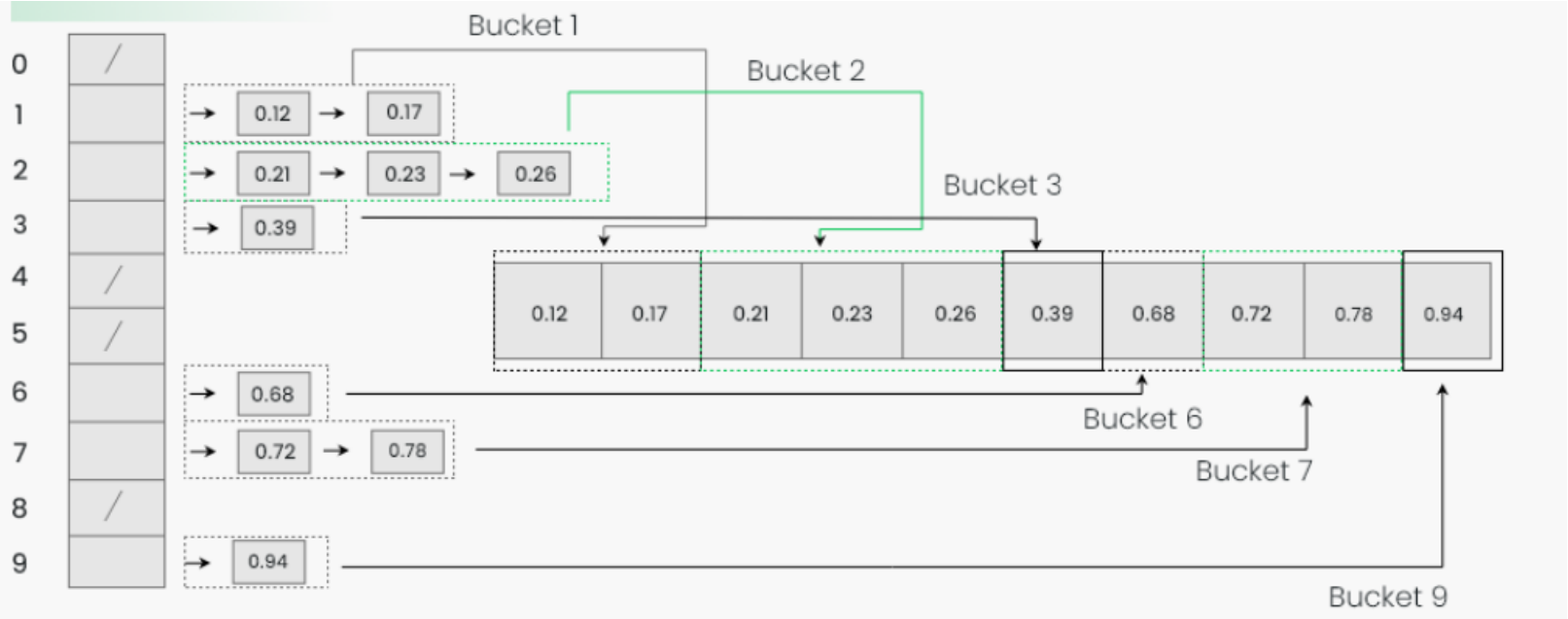


Each Bucket is treated a different Linked List and Sorted in ascending order

*Any stable sorting algorithm can be used for this purpose

Question 4

Step 4: Inserting buckets in ascending order into the resultent array



Question 4

Step 5 : Return the Sorted Array

0.12	0.17	0.21	0.23	0.26	0.39	0.68	0.72	0.78	0.94

Complexity Analysis of Bucket Sort Algorithm

- **Worst Case Time Complexity:** $O(n^2)$ The worst case happens when one bucket gets all the elements. In this case, we will be running insertion sort on all items which will make the time complexity as $O(n^2)$

Question 5

(a) (i) A priority queue is implemented as a Max-Heap. Initially, it has 5 elements. The level-order traversal (traversal and filling from left to right level by level) of the heap is: 10, 8, 5, 3, 2.

(ii) Two new elements 1 and 7 are inserted into the heap in the given order. What would be the level-order traversal of the heap after the insertion of the two new elements? Your answer should include insertion and fixing processes using step by step approach.

(b)(i) If one wants to delete the root value from the max-heap created in part (a)(ii) above. How could you eliminate the heap property violation which arises, using the top-down approach through fixing process without using an additional array?

(ii)How could you extend the methodology described in part b (i) above to sort all data values in the increasing order?

Question 6

- Explain the differences between the **Bucket Sort** algorithm and the **Radix Sort** algorithm.

Answer -Question 6

Aspect	Bucket Sort	Radix Sort
Basic Idea	Distributes the elements into a number of buckets, sorts each bucket (using another algorithm or recursively), and then concatenates the results.	Sorts numbers **digit by digit** (or bit by bit), starting from either the least significant digit (LSD) or most significant digit (MSD).
Type of Sorting	**Distribution sort** (based on value ranges).	**Digit-based sort** (based on positional representation of numbers).
Input Data	Works best when the input is **uniformly distributed** over a range (e.g., decimal fractions in [0, 1)).	Works best for **integers** or **strings** that can be broken into digits/characters.
Internal Sorting	Each bucket can be sorted using another algorithm (e.g., Insertion Sort or Quick Sort).	Uses a **stable sort** (like Counting Sort) to sort digits at each position.
Time Complexity	Average: (O(n + k)) (if elements are evenly distributed across buckets)Worst: (O(n^2)) (if all elements fall into one bucket).	(O(d \times (n + k))), where:(d) = number of digits(k) = range of each digit (e.g., 0–9 for decimal).
Stability	Depends on the sorting algorithm used inside buckets.	**Stable** if a stable algorithm is used for digit sorting (usually Counting Sort).
Example Use	Sorting floating-point numbers in [0, 1).	Sorting integers, phone numbers, or strings (e.g., “abc”, “bcd”).
Approach Summary	Divides data into *value-based buckets* → sorts each bucket.	Processes data *digit by digit* → sorts by each digit position.